CSA1702 ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

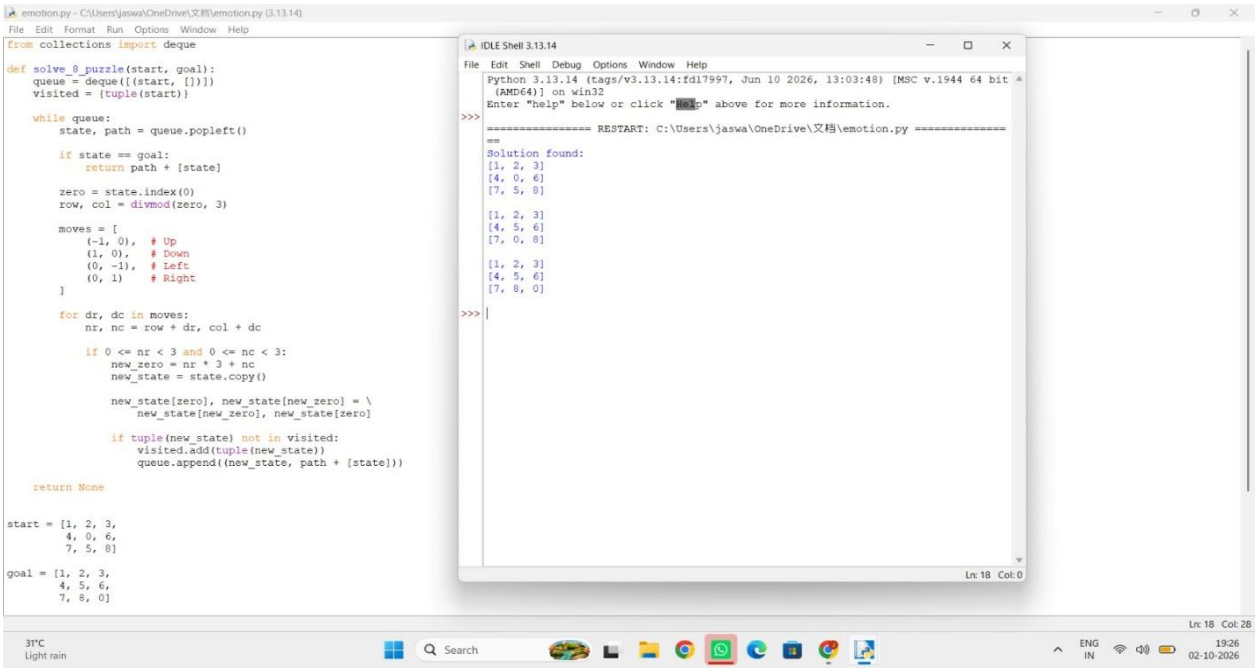
1. Python Program to Solve the 8-Puzzle Problem

Aim: To solve the 8-puzzle problem using Breadth-First Search (BFS) to find a path from the initial state to the goal state.

Algorithm:

1. Represent the puzzle as a 3×3 matrix.
2. Define the initial and goal states.
3. Locate the empty tile (0).
4. Generate possible moves in four directions.
5. Apply BFS to explore states.
6. Track visited states to avoid repetition.
7. Display the solution path when the goal is reached.

Python Code:



Result: Thus, the 8-puzzle problem was solved successfully using BFS.

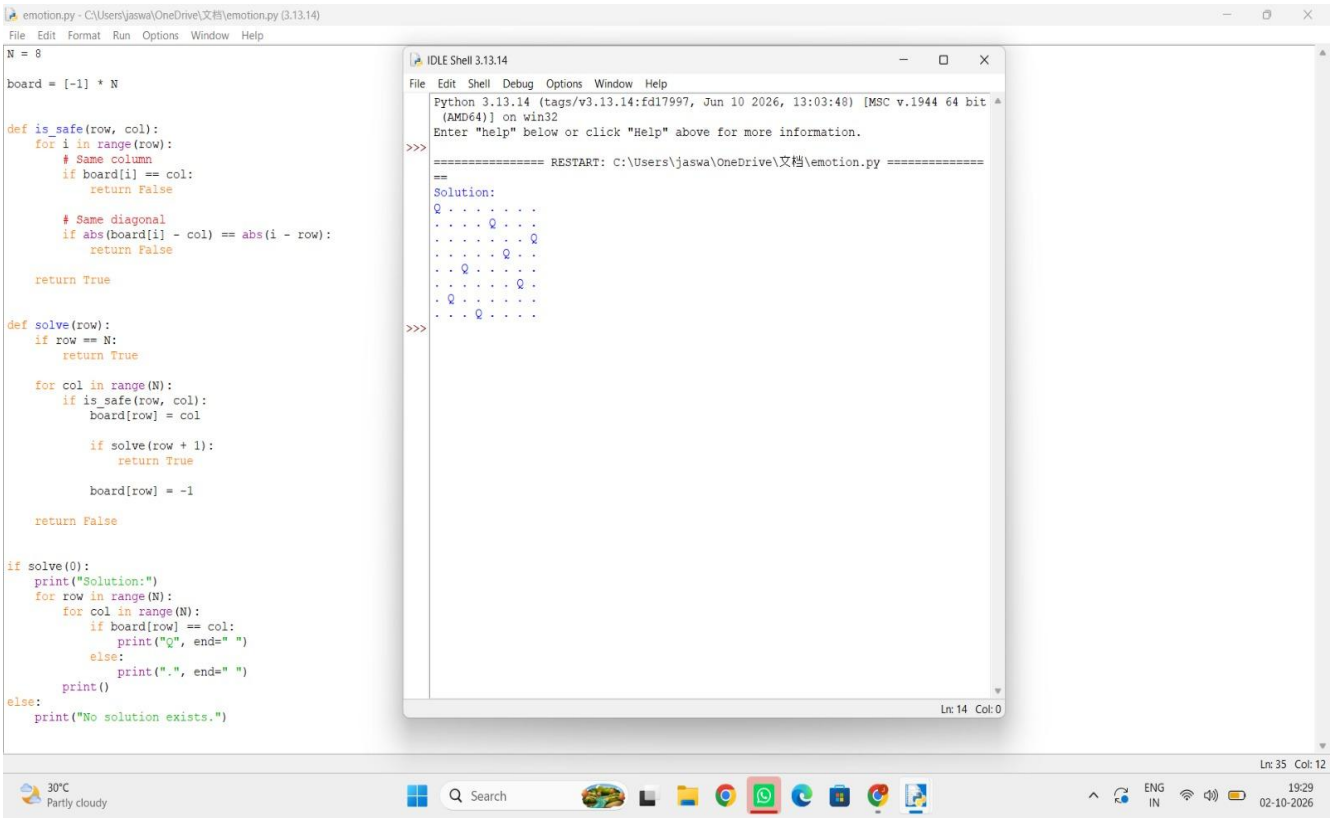
2. Python Program to Solve the 8-Queen Problem

Aim: To place eight queens on a chessboard so that no two queens attack each other.

Algorithm:

1. Create an 8×8 chessboard.
2. Place one queen in each row.
3. Check whether the column and diagonals are safe.
4. If safe, place the queen and move to the next row.
5. If no position is safe, backtrack.
6. Display the solution.

Python Code:



Result: Thus, the 8-Queen problem was solved successfully using backtracking.

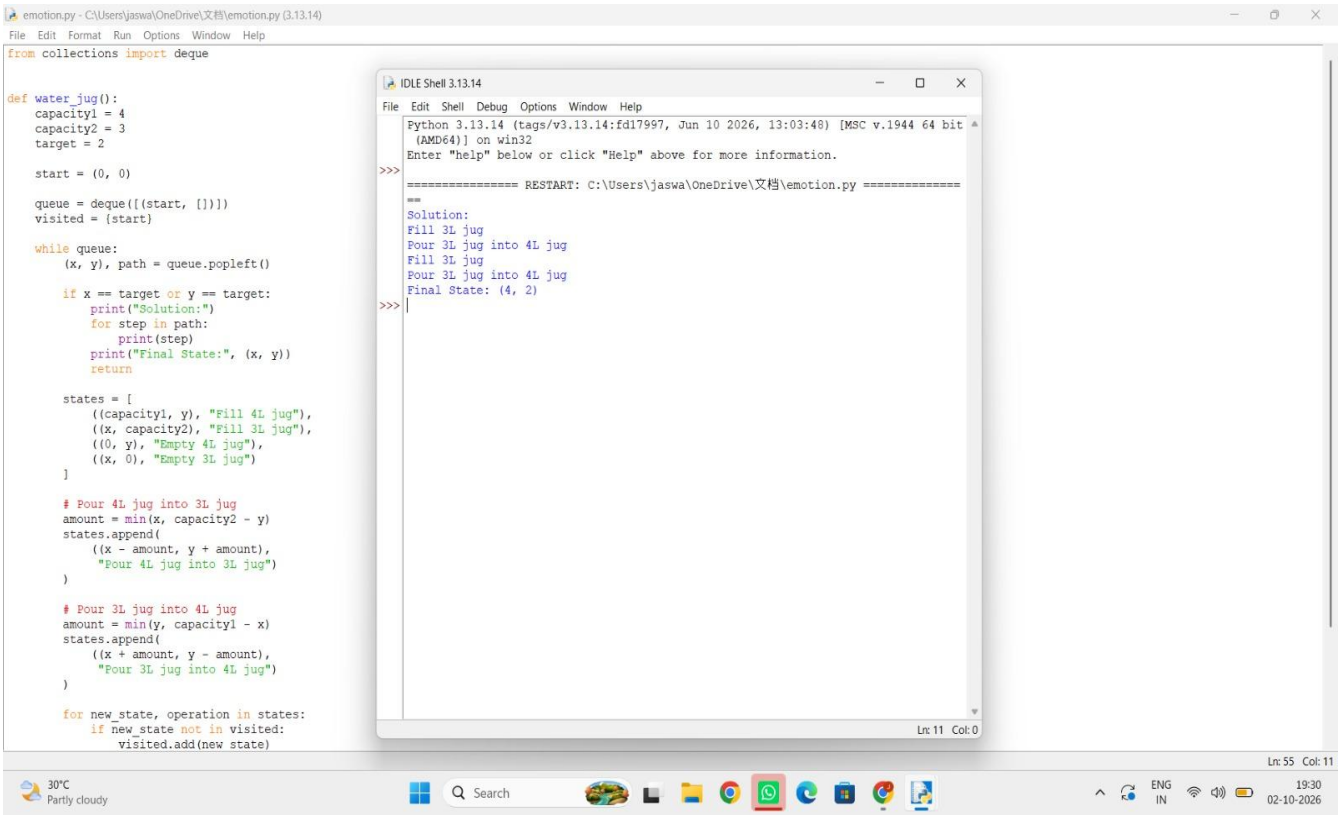
3. Python Program for the Water Jug Problem

Aim: To measure a target quantity of water using two jugs of different capacities.

Algorithm:

1. Define the capacities of the two jugs.
2. Define the target quantity.
3. Start with both jugs empty.
4. Generate states using filling, emptying, and pouring operations.
5. Use BFS to find the target state.
6. Display the sequence of states.

Python Code:



Result: Thus, the Water Jug Problem was solved successfully using BFS.

4. Python Program for the Crypt-Arithmetic Problem

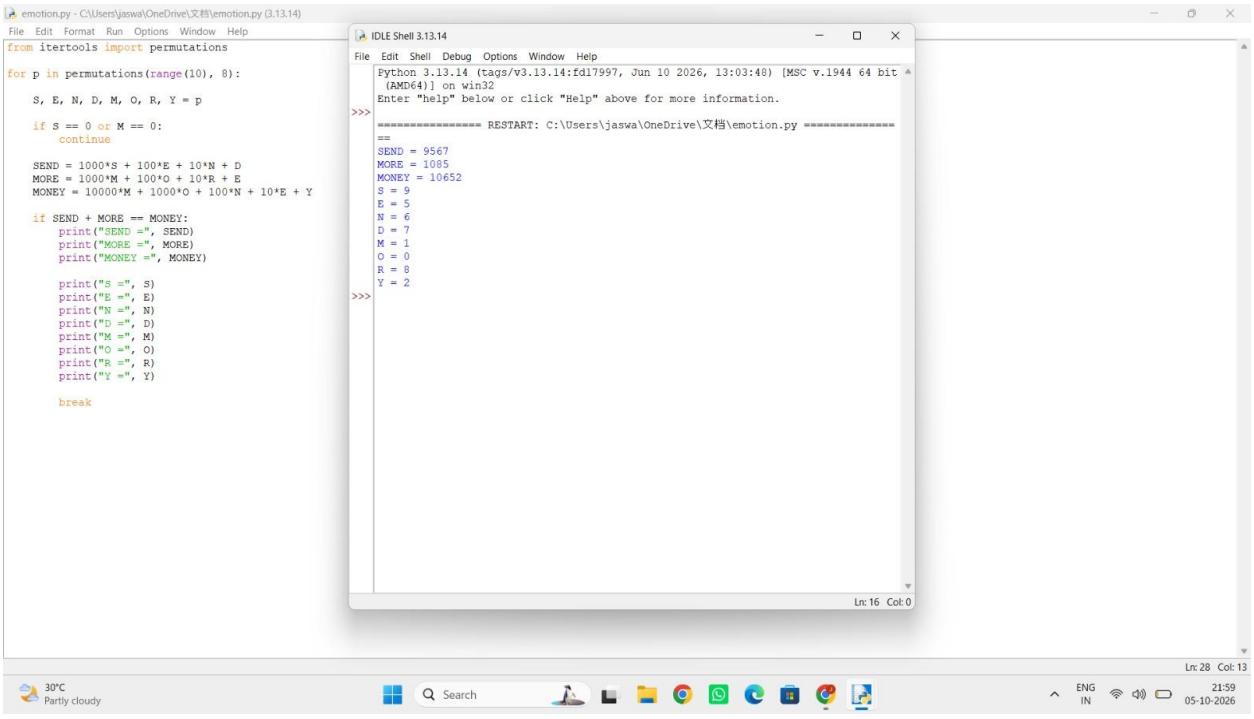
Aim: To solve a crypt-arithmetic puzzle by assigning unique digits to letters.

Example: SEND + MORE = MONEY

Algorithm:

1. Identify the unique letters.
2. Assign different digits to each letter.
3. Ensure that the first letters are not zero.
4. Evaluate the numerical equation.
5. Display the digit assignment when the equation is satisfied.

Python Code:



Result: Thus, the crypt-arithmetic problem was solved successfully by assigning unique digits to letters.

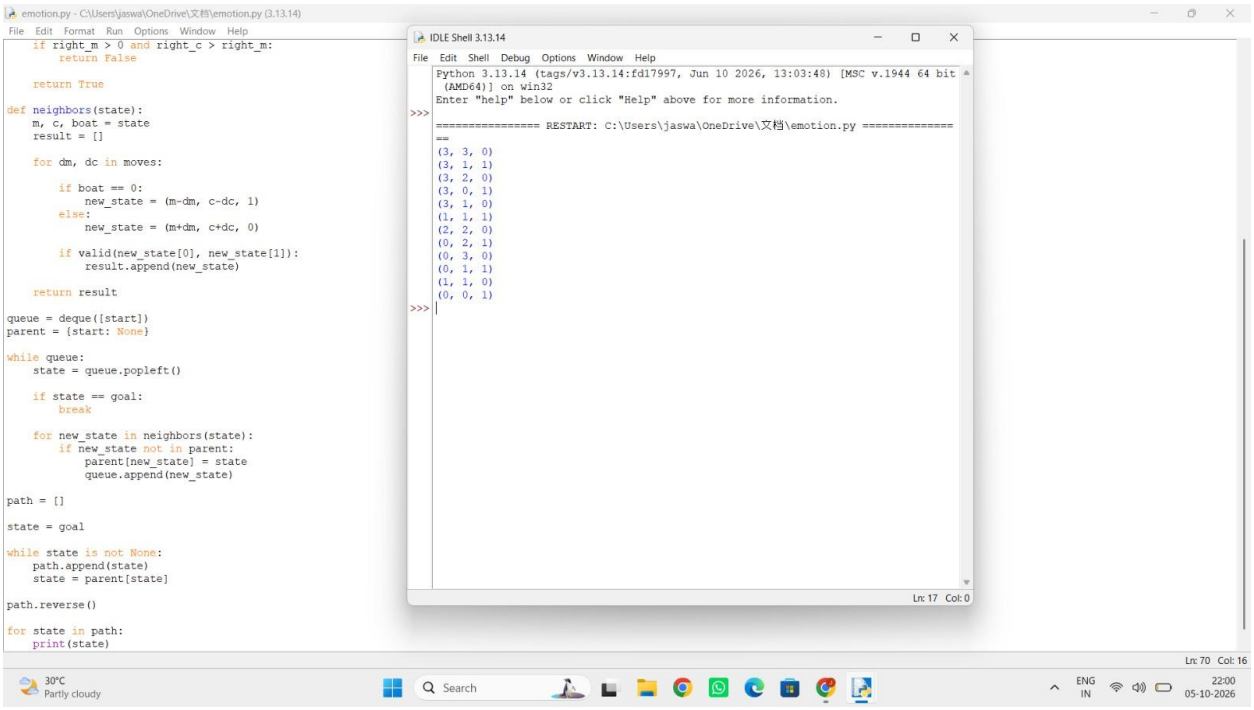
5. Python Program for the Missionaries and Cannibals Problem

Aim: To transport three missionaries and three cannibals across a river without allowing cannibals to outnumber missionaries on either bank where missionaries are present.

Algorithm:

1. Represent the number of missionaries and cannibals on the left bank and the boat's position.
2. Start with three missionaries and three cannibals on the left bank.
3. Allow the boat to carry one or two people.
4. Check that the missionaries are never outnumbered on either bank.
5. Use BFS to find a valid sequence of crossings.
6. Display the solution.

Python Code:



Result: Thus, the Missionaries and Cannibals Problem was solved successfully using BFS.

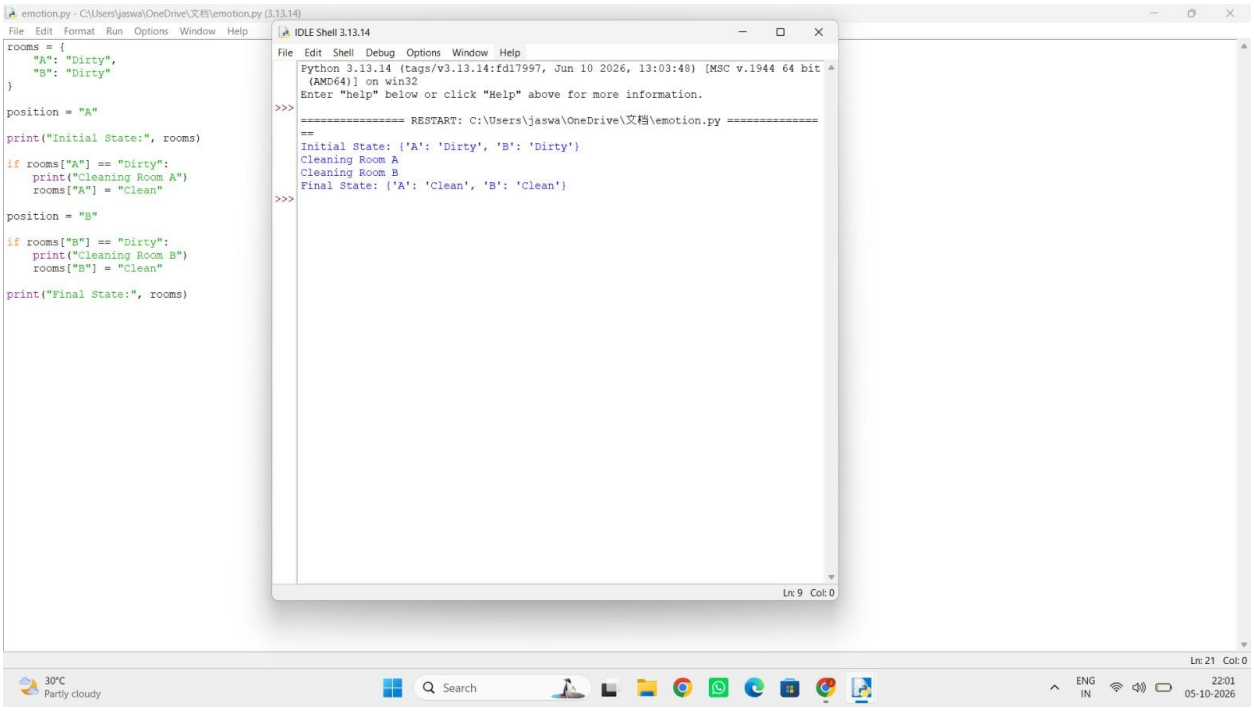
6. Python Program for the Vacuum Cleaner Problem

Aim: To implement a simple vacuum cleaner agent that cleans dirty rooms.

Algorithm:

1. Represent two rooms, A and B.
2. Mark each room as clean or dirty.
3. Place the vacuum cleaner in room A.
4. Clean the current room if it is dirty.
5. Move to the other room and clean it if required.
6. Display the final room conditions.

Python Code:



Result: Thus, the Vacuum Cleaner Problem was solved successfully by cleaning both rooms.

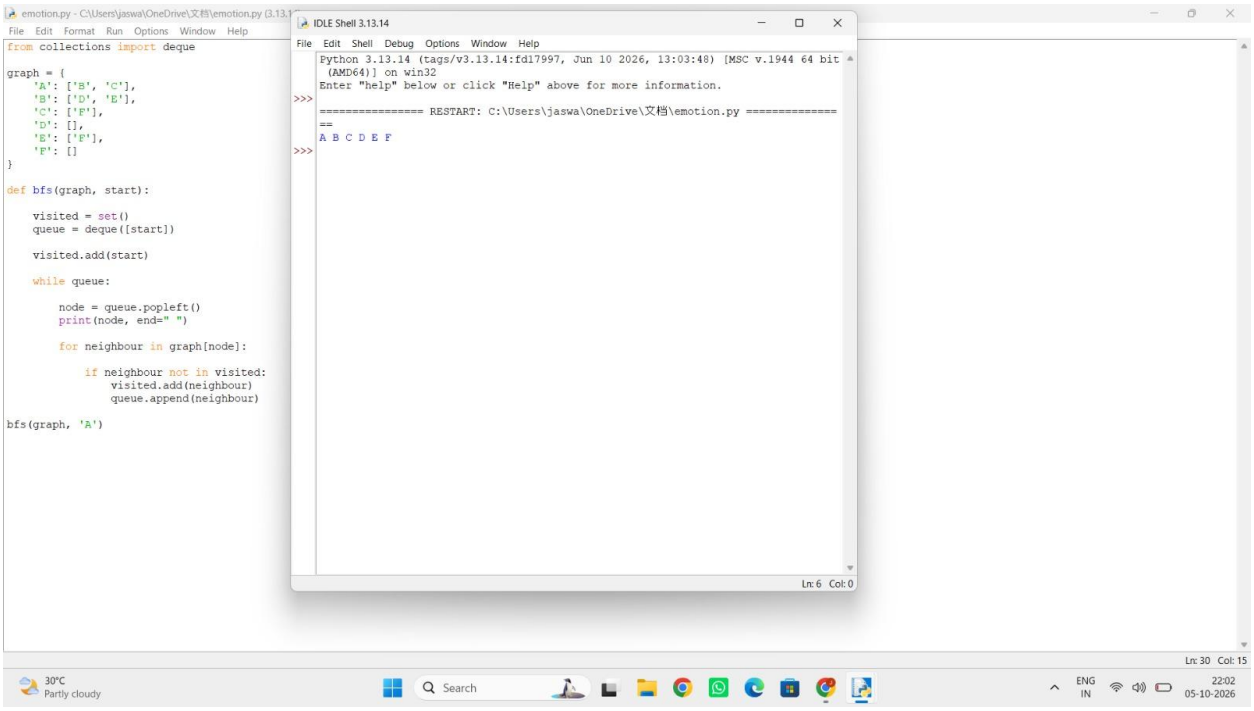
7. Python Program to Implement Breadth-First Search (BFS)

Aim: To traverse a graph using Breadth-First Search.

Algorithm:

1. Create a graph using an adjacency list.
2. Initialize a queue with the starting node.
3. Mark the starting node as visited.
4. Remove a node from the queue and print it.
5. Add its unvisited adjacent nodes to the queue.
6. Repeat until the queue is empty.

Python Code:



Result: Thus, graph traversal was performed successfully using BFS.

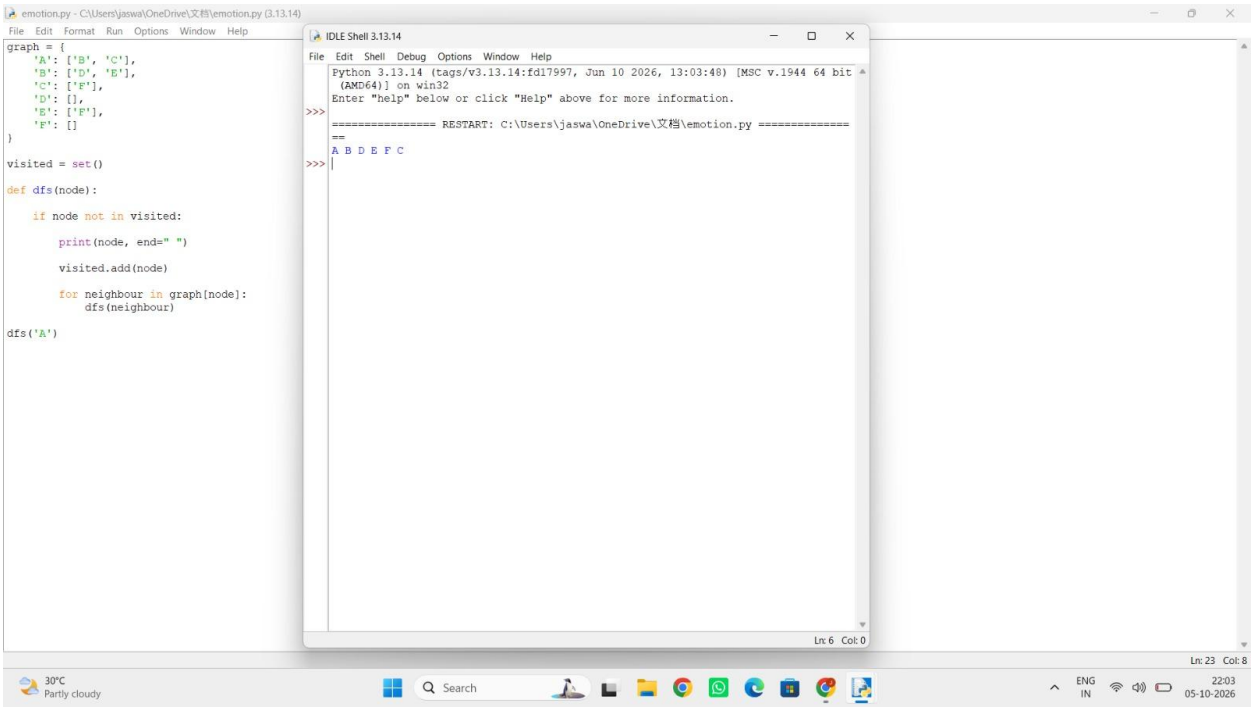
8. Python Program to Implement Depth-First Search (DFS)

Aim: To traverse a graph using Depth-First Search.

Algorithm:

1. Create a graph using an adjacency list.
2. Start from the selected node.
3. Mark the node as visited and print it.
4. Visit an unvisited adjacent node recursively.
5. Continue until all reachable nodes are visited.

Python Code:



Result: Thus, graph traversal was performed successfully using DFS.

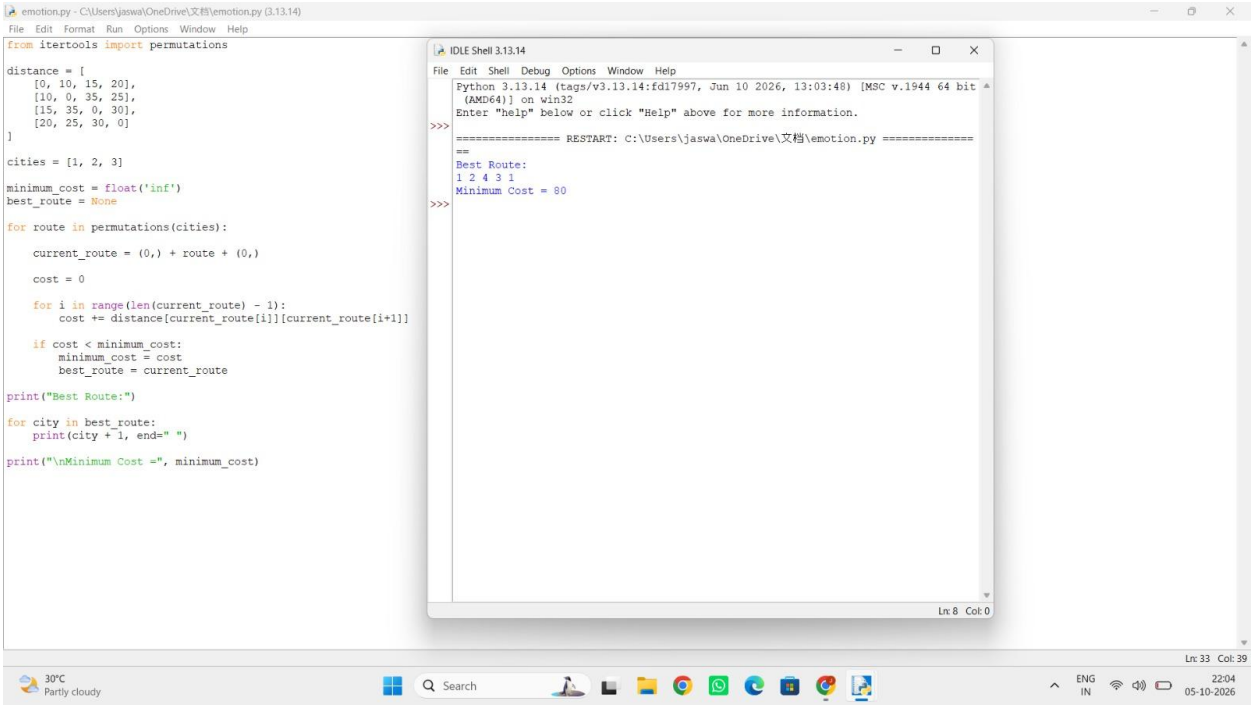
9. Python Program for the Travelling Salesman Problem (TSP)

Aim: To find the minimum-cost route that visits every city exactly once and returns to the starting city.

Algorithm:

1. Represent the distances between cities using a matrix.
2. Fix the starting city.
3. Generate all possible orders of the remaining cities.
4. Calculate the total distance of each route, including the return journey.
5. Select and display the route with the minimum distance.

Python Code:



Result: Thus, the Travelling Salesman Problem was solved successfully by finding the minimum-cost route.

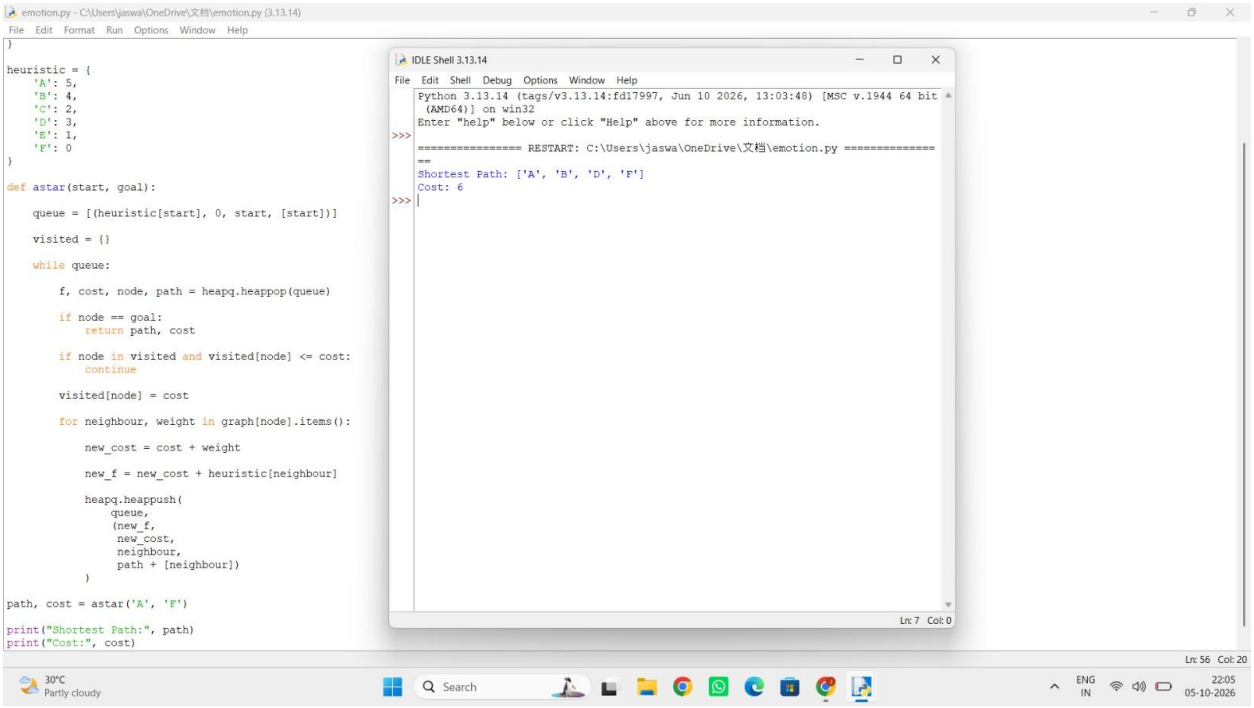
10. Python Program to Implement the A* Algorithm

Aim: To find the shortest path to the goal state using the A* search algorithm.

Algorithm:

1. Define the initial and goal states.
2. Use a priority queue to select the state with the lowest estimated cost.
3. Calculate \(f(n)=g(n)+h(n)\), where \(g(n)\) is the cost so far and \(h(n)\) is the Manhattan distance heuristic.
4. Generate all valid neighboring states.
5. Update their costs and paths.
6. Continue until the goal state is reached.

Python Code:



Result: Thus, the A* algorithm was implemented successfully to find the solution path.